Syllabus

General Description

CS 3500 teaches a rigorous approach to object-oriented programming and design, with an emphasis on abstraction, modularity, and code reuse as applied to the building and understanding of large-scale systems. We will explore the basic mechanisms and concepts of object-oriented programming: object, class, message, method, interface, encapsulation, polymorphism, and inheritance. Students will gain hands-on experience with tools and techniques that facilitate the creation and maintenance of applications using the Java programming language.

Prerequisites

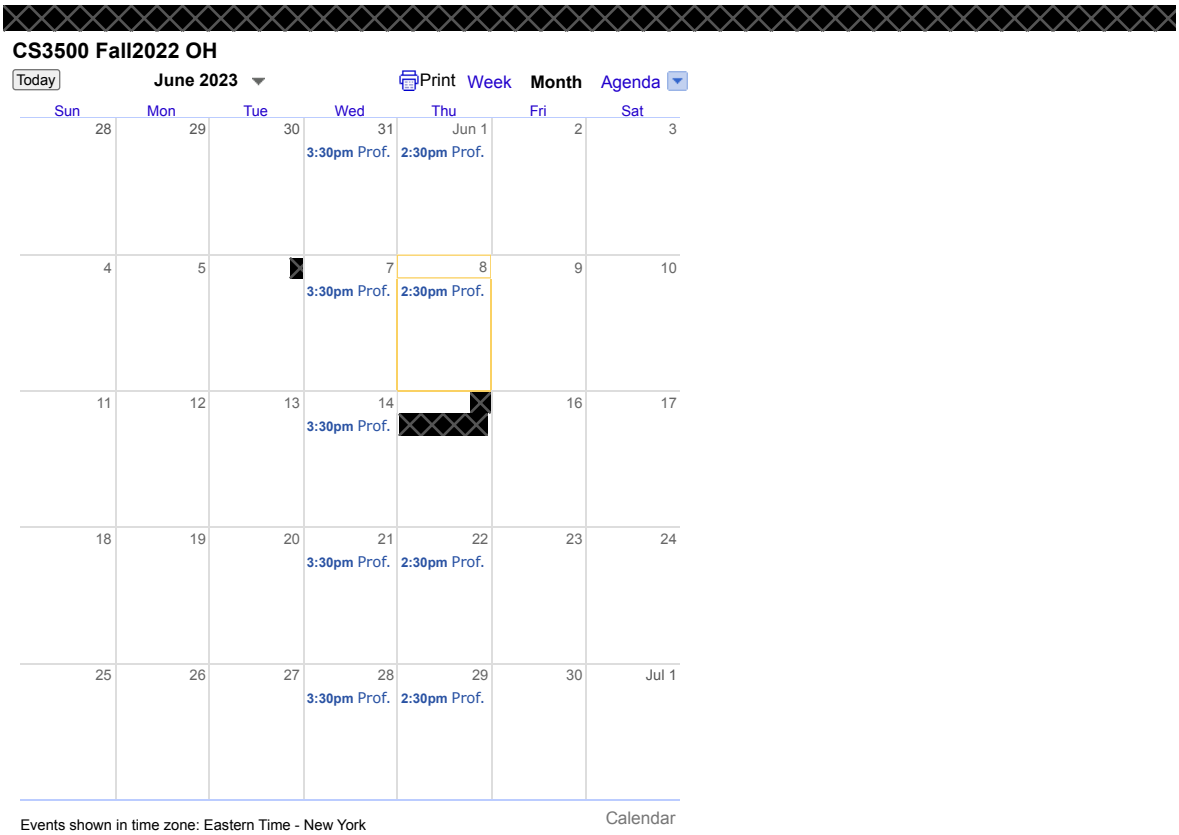
This course assumes familiarity with programming in the style of *How to Design Programs* , and basic knowledge of the Java programming language as introduced in CS 2510.

Outcomes

Upon completion of this course, a student will be able to:

- Design an object-oriented solution using abstraction, encapsulation and reuse appropriately for a given problem statement
- Apply relevant design patterns to a given design problem
- Implement an object-oriented design in the Java programming language
- Design and implement unit tests for a component or set of components
- Work with another student on programming and design activities
- Read and understand given files/syllabus/code/design diagrams well enough to use and extend it

Course Staff



Materials

Software

For programming assignments, we will use Java 11. **You should download and install the Java SE Development Kit, version 11 (https://www.oracle.com/technetwork/java/javase/downloads/jdk11-downloads-5066655.html) from Oracle. You would need to create a free Oracle account in order to this. If you do not wish to create an account or the process above doesn't work, you can perform the download from this mirror page. (https://www.techspot.com/downloads/5553-java-jdk.html)**

The supported IDE (integrated development environment) for the course is **IntelliJ IDEA (http://www.jetbrains.com/idea/)**. This is the IDE that the instructor uses in lecture, and we may occasionally give instructions for how to perform particular tasks in IDEA. You are free to use a different IDE, but we may not be able to help you if you run into trouble. IntelliJ IDEA Community Edition may be downloaded free of charge, and you can also license the Ultimate Edition. Please go to the **Jetbrains website to get a free academic license to the Ultimate Edition (https://www.jetbrains.com/community/education/#students)**.

If you are having trouble setting up IntelliJ, **we have a video demonstrating how to configure your environment (https://ccs.neu.edu/home/ashesh/IntelliJ-setup.mp4)**.

Books

There is no required textbook, but you may find these books useful.

- Joshua Bloch, **Effective Java** **Second Edition (https://www.amazon.com/Effective-Java-Edition-Joshua-Bloch/dp/0321356683)** or **Third Edition (https://www.amazon.com/Effective-Java-Joshua-Bloch/dp/0134685997/)** (Highly recommended. All Java programmers should read and understand this book.)
- Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, **Design Patterns: Elements of Reusable Object-Oriented Software (http://www.amazon.com/Design-Patterns-Elements-Reusable-Object-Oriented/dp/0201633612)** Addison-Wesley, 1995. (The standard reference for object-oriented design patterns.)
- Eric Freeman and Elisabeth Robson, with Kathy Sierra and Bert Bates, **Head First Design Patterns: A Brain-Friendly Guide (https://www.amazon.com/Head-First-Design-Patterns-Brain-Friendly/dp/0596007124)** O'Reilly, 2014. (An entertaining guide to object-oriented design patterns, with examples, illustrations, and exercises.) **Full source code for the examples in Head First Design Patterns is available on GitHub (https://github.com/bethrobson/Head-First-Design-Patterns)**
- Peter Sestoft, **Java Precisely, Second Edition (http://mitpress.mit.edu/books/java-precisely)**, MIT, 2005. (May be especially useful if your Java skills are weak.)

Online resources

- **Design Principles Master List (https://northeastern.instructure.com/courses/119265/pages/Design%20Principles%20Master%20List)** : a distillation of the key design concepts taught in the course, in list form.
- **Piazza discussion board (http://piazza.com/northeastern/fall2022/cs3500/home)** ---ask questions here!
- **Google's Java style guide--follow this! (https://google.github.io/styleguide/javaguide.html)** Your grade will reflect your adherence to good style.}
- **IntelliJ configuration file to automate Google style (https://raw.githubusercontent.com/google/styleguide/gh-pages/intellij-java-google-style.xml)** -- download this file and load it into IntelliJ to have your code formatted automatically.
- **Java API Reference (https://docs.oracle.com/en/java/javase/11/docs/api/index.html)** Standard Edition, version 11
- **Java Tutorials (http://docs.oracle.com/javase/tutorial/index.html)**
- **An interactive tutorial on Git (http://pcottle.github.io/learnGitBranching/)**

Testing

Testing your code is sufficiently important that we've devoted an entire page to it. Please read **these notes (https://northeastern.instructure.com/courses/119265/pages/How%20To%20Think%20About%20Testing)** , for each and every assignment you work on.

Course policies

Collaboration and academic integrity

You may not collaborate with anyone on any of the exams. You may not use any electronic tools, including phones, tablets, netbooks, laptops, desktop computers, etc. If in doubt, ask a member of the course staff.

Some homework assignments will be completed with an assigned partner, and some may involve a larger team (TBD). You must collaborate with your assigned partner or team, as specified, on homework assignments. You may request help from any staff member on homework. (When you are working with a partner, we strongly recommend that you request help with your partner.) You may use the **Piazza bulletin board (http://piazza.com/northeastern/fall2022/cs3500/home)** to ask questions regarding assignments, so long as your questions (and answers) do not reveal information regarding solutions. You may not get any help from anyone else on a homework assignment; all material submitted must be your own. If in doubt, ask a member of the course staff.

Providing illicit help to another student is also cheating, and will be punished the same as receiving illicit help. It is your responsibility to safeguard your own work.

A subtler form of cheating, but nevertheless illegal, is *self-plagiarism* . This is when you copy and submit code that you developed when you took CS 3500 previously for credit again. This is illegal if done for your group assignments, because the code you are attempting to submit has not been written just by you and your partner in the current semester.

Students who cheat will be reported to the university's office on academic integrity and penalized by the course staff, at our discretion, **up to and including failing the course** .

If you are unclear on any of these policies, please ask a member of the course staff.

Homework

In general, you should submit your homework according to the instructions on the web page for the individual assignments.

Using the handin server

You will submit your homework using `https://handins.ccs.neu.edu` **(https://handins.ccs.neu.edu)** . We have written a **how-to guide for using the handin server (http://www.ccis.northeastern.edu/home/blerner/handin-server/handin-server-guide.html)** , if you are unfamiliar with it. You will need an *active* Khoury account to log in and submit on this server. If you are a CS major or have taken a CS class before, you should have one already. For all account services (including creating an account, retrieving a lost username or password), please go to **Getting Started (https://www.khoury.northeastern.edu/life-at-khoury/systems/getting-started/)** . **Make sure you can log in to the handin server, and register for your section of the course, before the first homework is assigned.**

Unlike previous courses you may have taken such as CS 2510, you will usually need to submit multiple files for your assignments, rather than a single file. You should therefore add all your files to a `.zip` or `.tar` file, and submit that archive file. Note that:

- Java is **very picky** about directory organization. Your archive files should therefore contain your `src/` and `test` directories *only* , and not have them nested within some other directory by mistake, or include your compilation directory unnecessarily.
- The handin server is very picky about the sorts of archives you send it. Send it only `.zip` files, `.tar` files, or `.tar.gz` files -- other archive formats will be rejected. So will archives larger than 5MB or 100 files and folders: if the server rejects your archive, check that you didn't include unneeded files.

Additionally, every homework where you write code will be followed one day later by a *self-evaluation* assignment. Only one partner (for team assignments) needs to complete the self-eval, and it will count for both partners.

Submission troubles

If you have trouble submitting to the server and you have time before the deadline, please wait few minutes and try again; it may also be worth checking on Piazza to find out whether other students are experiencing similar difficulties. If upon retrying you still cannot submit, email your professor.

Your email should have the subject "HW *N* submission problems" (where *N* is the appropriate homework number), and you should attach the ZIP file **exactly** as you would have submitted it to the server.

Late days and late work

Each student gets four free, no-questions-asked late days for the term. Two of these days can be used for individual assignments, and two will be for group assignments. A late day extends the deadline of an assignment by 24 hours without any grade penalty.

The purpose of late days is make the extension process fair and transparent by getting the instructors out of the extension-granting business entirely. Instead, when you need an extension, you can take one—provided you have a late day remaining. **If you do not have any late days remaining, there is no opportunity to submit late for a penalty.**

Late days are expected to be used for reasons like (but not limited to) illness, submitting again to improve score beyond the deadline, other activities causing a delay in submission, etc. Late days are **not in addition to** other things that may come up in the semester. The instructors will work with you if you require additional accommodations for health or other unexpected emergencies, but please do not bank on getting extensions while saving your late days.

To use a late day, just submit the homework as normal. The server will keep track of the number of used late days, and will prevent you from submitting more than one day late to any assignment, or late at all if you've used all your late days. Conserve your late days carefully. However unused late days cannot be cashed in for extra credit at the end of the semester.

No more than one late day may be used on any one homework. You may not look at and must avoid gaining knowledge of the self-evaluation questions until you have submitted your late assignment. Late days cannot be divided fractionally, but must be used whole. Late days cannot be transferred to or shared with a partner, so in order to take an extension both you and your partner must have sufficient late days remaining. Choose your partners carefully.

Using a late day to submit your files does not automatically grant you a late day for the self-eval: it will remain due at the normal time.

Grades

Your grade will be based on your performance on the assignments (60%), labs (5%), and the exams (15%, 20%). Material for examinations will be cumulative.

The grades will computed on an absolute basis: **there will be no overall curving** . The instructor may choose to curve an individual homework or exam, but please do not bank on such a chance.

The mapping of raw point totals to letter grades is given below. Please note that these grade boundaries may move slightly at the discretion of the instructor, but the grade boundary for A is unlikely to change. Grades are not rounded: for instance, earning a 92.5% does not imply we will round up to a 93% and hence to an A.}

Range	Letter Grade
A	93%
A-	90%
B+	87%
B	83%
B-	80%
C+	77%
C	73%
C-	70%
D+	67%
D	63%
D-	60%
F	0%